QUANTUMSENTINEL · QISKIT FALL FEST 2026 · IEM KOLKATA × IBM QUANTUM

Qampaign: a leak-free, three-stage quantum-classical pipeline for a bank deposit campaign

Predict who will subscribe, select exactly six customers under budget and capacity, and simulate the chance that at least three subscribe. Code, data, results and notebook: **github.com/souvikbasuroy/Qampaign** (MIT licence).

1. The challenge and what we set out to do

One bank runs one campaign. Contacting every customer is wasteful, and picking only the likeliest customers ignores costs and limits. The task is a single connected pipeline in which **Stage 2 and Stage 3 consume our own Stage 1 probabilities, so errors carry forward**.

1 · Predict
300 labelled customers for training, 100 unlabelled for scoring. Keep the top 10.

→

2 · Select
Exactly 6 of the 10, maximising expected net value within budget (25) and capacity (22).

→

3 · Simulate
A 6-qubit circuit gives the chance that at least 3 of the 6 subscribe.

Project aims

- **A1 · Honest prediction.** Every scaler, encoder, reducer, feature choice, hyper-parameter and calibration step is fitted on training data only, inside nested repeated cross-validation. The test set is scored once.
- **A2 · Fair classical-vs-quantum comparison.** A regularised logistic baseline and a ZZ quantum-kernel SVC get identical folds and an identical 40-setting tuning budget; the winner is picked by out-of-fold log loss.
- **A3 · A real quantum formulation of the selection problem,** checked against exact enumeration: a slack-variable QUBO, and a constraint-preserving Dicke-state/XY-mixer QAOA that needs no slack qubits.
- **A4 · A faithful simulation stage,** one qubit per customer with an exact Poisson-binomial reference, a Monte Carlo cross-check, a synthetic-noise run and an uncertainty stress test.
- **A5 · Transparent claims.** Classical and quantum work are reported separately, every limitation is stated, and no quantum advantage is claimed.

2. Headline results

Stage	Result	Value
1 · Predict	Selected model (lowest nested-CV log loss)	Logistic regression + Platt calibration; ROC-AUC 0.619, PR-AUC 0.226, Precision@10 0.600
1 · Predict	Quantum ZZ-kernel SVC	ROC-AUC 0.608, PR-AUC 0.149; loses on PR-AUC and probability quality
2 · Select	Six customers (exact optimum, submitted)	**C0081, C0031, C0079, C0222, C0087, C0256**; objective **16.122**; cost 13/25; capacity 11/22
2 · Select	QAOA vs exact	Optimality gap 0; best-of-80 run puts 37.9% of shots on the optimum (chance 0.48%, median run 0.68%)
3 · Simulate	P(at least 3 of 6 subscribe)	**Exact 0.181754**; 20,000-shot Aer 0.1810; 1M-draw Monte Carlo 0.181689; noisy 0.20195, readout-mitigated 0.19038

Data provided: README, Stage1_Train (300 rows, 35 subscribers, base rate 11.67%), Stage1_Test (100), Stage2_Data (400 rows with contact cost, expected deposit value, capacity units, segment) and Stage2_Constraints (N=10 candidates, K=6 selected, budget C=25, capacity H=22). The workbook hash is recorded in *data/WORKBOOK_SHA256.txt*; the loader validates sheets, row counts, unique IDs, yes/no labels and rejects test labels and the *duration* column (known only after a call).

Repo reference: *main.py* → *src/customer_pipeline/pipeline.py* (orchestrator, MLflow), *workbook.py* (validation), *settings.yaml* (all settings).

STAGE 1 · PREDICT · METHOD

Predicting who will subscribe, without touching the test set

Features

Transforms are deterministic, so nothing is learned from held-out rows. Numeric columns are standardised and categoricals one-hot encoded inside each training fold.

$$\text{balance_log_signed} = \operatorname{sign}(b)\ln(1+|b|), \qquad \text{campaign_log} = \ln(1+\max(c,0))$$

Three flags are added: previously contacted (pdays ≥ 0), previous positive (previous > 0) and *poutcome* = success.

Two model families, equal budgets

Classical baseline. L2-regularised logistic regression. Search: 2 feature sets (all / compact) × C ∈ {0.03, 0.1, 0.3, 1.0} × 5 class weights = 40 settings, scored by inner log loss.

$$P(y=1 \mid x) = \sigma(w^{\top}x + b)$$

Quantum-kernel model. Compact features → truncated SVD to q ∈ {4, 6} components (one per qubit) → min-max scaling to [0, π] → scale s ∈ {0.5, 1.0} with clipping → Qiskit *ZZFeatureMap* (linear entanglement, reps ∈ {1, 2}). The kernel is the exact state-fidelity (statevector, 0 shots) and feeds an SVC with a precomputed kernel, C ∈ {0.01 … 100}, balanced class weights: 2×2×2×5 = 40 settings, scored by inner ROC-AUC. Ranking uses the SVC decision function, never its built-in probability.

$$K(x,x') = \left|\langle \phi(x) \mid \phi(x') \rangle\right|^2$$

Validation, calibration and model choice

Nested repeated stratified CV: outer 5 repeats × 5 folds, inner 3-fold tuning, identical folds for both families. Platt (sigmoid) calibration is fitted in each outer-training fold. The family with the lowest mean outer log loss wins; it is then refitted on all 300 rows and predicts the 100 test customers once. Ties in the ranking go to the lower customer_id.

Metrics and uncertainty

$$\text{Brier} = \frac{1}{n}\sum_i (p_i - y_i)^2, \qquad \text{LogLoss} = -\frac{1}{n}\sum_i [y_i \ln p_i + (1-y_i)\ln(1-p_i)]$$

$$\text{Precision@}k = \frac{\text{subscribers in top } k}{k}, \qquad \text{Recall@}k = \frac{\text{subscribers in top } k}{35}$$

- **Repeated-CV interval:** mean ± t(0.975, 4)·s/√5 across the five outer repeats. It understates uncertainty because folds overlap.
- **OOF bootstrap interval:** 1,000 stratified resamples of pooled out-of-fold (label, probability) pairs, 2.5th to 97.5th percentile. This is the more honest interval.
- **Kernel-target alignment** (centred kernel K_c vs target T_c built from $yy^{\top}$, $y \in \{-1, +1\}$) measures whether the quantum kernel separates the classes:

$$A = \frac{\langle K_c, T_c \rangle_F}{\|K_c\|_F \, \|T_c\|_F}$$

Leakage guards in code

validate_stage1_inputs raises if a test label or *duration* appears; unit tests verify that model choice uses only training-CV log loss and that both families have equal 40-setting budgets. Both classical and quantum results are written to separate rows of *results/stage1_cv_metrics.csv*.

Repo reference: *src/customer_pipeline/stage1.py* (*CampaignFeatureEngineer*, *FastZZKernel*, *QuantumKernelSVC*, *_tune_logistic*, *_tune_quantum*, *run_stage1*); *src/eval_extras/metrics.py* (PR-AUC); *tests/test_customer_stage1.py*.

STAGE 1 · PREDICT · RESULTS

Classical wins; the quantum kernel adds nothing here

Metric (mean, 95% intervals)	Logistic (selected)	ZZ quantum-kernel SVC
ROC-AUC	0.619 (CV 0.577–0.661; boot 0.515–0.717)	0.608 (CV 0.582–0.633; boot 0.535–0.704)
PR-AUC	0.226 (CV 0.193–0.259; boot 0.137–0.365)	0.149 (CV 0.131–0.167; boot 0.117–0.188)
Precision@10 / @20 / @30	0.600 / 0.310 / 0.213	0.140 / 0.160 / 0.147
Recall@10 / @20 / @30	0.171 / 0.177 / 0.183	0.040 / 0.091 / 0.126
Brier score (lower is better)	0.097 (boot 0.089–0.104)	0.102 (boot 0.100–0.103)
Log loss (lower is better)	0.342 (boot 0.317–0.363)	0.354 (boot 0.343–0.361)

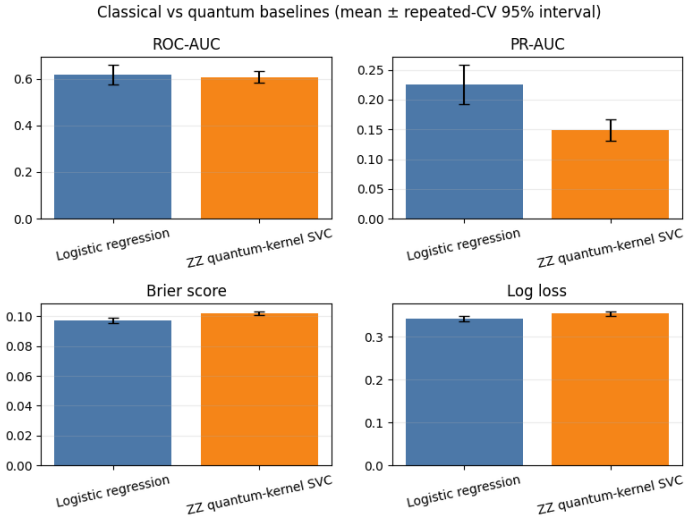


Fig. 1. Means with repeated-CV 95% whiskers under identical folds. ROC-AUC overlaps; logistic is clearly ahead on PR-AUC and has lower Brier and log loss.

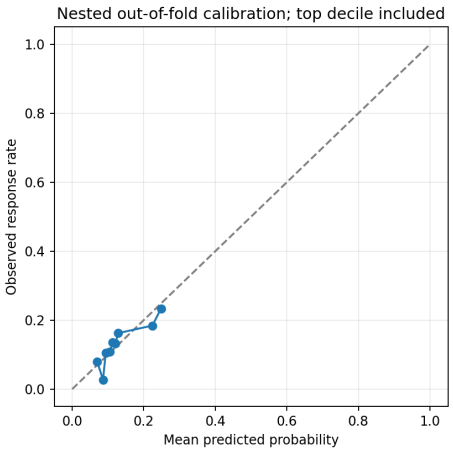


Fig. 2. Out-of-fold calibration of the selected model (pooled quantile bins plus a top-decile bin). Points sit near the diagonal; each bin has only about 37 customers.

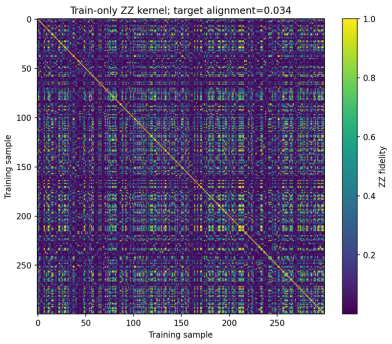


Fig. 3. Train-only ZZ fidelity kernel (300×300). Useful kernels show block structure; this one is diffuse, matching alignment 0.034.

What the results mean

- **Model choice.** The logistic family has the lower out-of-fold log loss (0.342 vs 0.354) and a PR-AUC about 50% higher. The quantum kernel is competitive only on ROC-AUC, within noise.
- **Precision@10 = 0.600 is one feature.** The same ten customers rank first in every repeat: the ten training customers with *poutcome* = success, six of whom subscribed. The score has zero spread for that reason.
- **Raw vs calibrated quantum score.** Fold-level ROC-AUC is 0.614 on the raw decision score and 0.607 after Platt calibration, so calibration does not hurt ranking.
- **Test predictions.** Only three customers stand out: C0081 (0.398), C0031 (0.372), C0079 (0.332). Ranks 4–10 lie between 0.153 and 0.168; rank 11 (C0397, 0.1515) is just 0.0013 below rank 10 (C0256, 0.1528). Swapping them gives C0397 in the six and raises the objective from 16.122 to 17.306, so the boundary is fragile. This is a robustness note only; the submission is unchanged.
- **Range check.** Test top-ten probabilities (0.153–0.398) lie inside the calibrated out-of-fold range (0.052–0.536).

Repo reference: *results/stage1_cv_metrics.csv*, *stage1_calibration_curve.csv*, *stage1_quantum_kernel_matrix.csv*, *stage1_quantum_qubit_feature_order.csv*; root file *stage1_predictions.csv* (customer_id, predicted_probability, rank, is_top_10).

STAGE 2 · SELECT · FORMULATION

Choosing exactly six customers as a binary optimisation problem

For each of the top-10 candidates, let p_i be the Stage 1 probability, D_i the expected deposit value, c_i the contact cost and h_i the capacity units. Decision bit $x_i = 1$ means "contact customer i" (bit i is rank i+1 in the top-10 list).

$$v_i = p_i D_i - c_i, \qquad \max_{x \in \{0,1\}^{10}} \sum_i v_i x_i \quad \text{s.t.} \quad \sum_i x_i = 6, \quad \sum_i c_i x_i \le 25, \quad \sum_i h_i x_i \le 22$$

Valid means exactly six unique IDs from our own top 10 within both limits; *validate_selection* enforces this and the pipeline stops with an error otherwise.

Formulation 1 · slack-variable QUBO (20 qubits)

Inequalities become equalities with binary slack bits: 5 for the budget and 5 for the capacity, plus 10 decision bits. With penalty P = 10,000, and the Ising map x = (1 − Z)/2:

$$E = -\sum_i v_i x_i + P\left[\left(\sum_i x_i - 6\right)^2 + \left(\sum_i c_i x_i + \sum_j 2^j s_j - 25\right)^2 + \left(\sum_i h_i x_i + \sum_j 2^j t_j - 22\right)^2\right]$$

This encoding is retained as a verified alternative: the code minimises the slack analytically for every decision vector and **asserts that the QUBO minimum equals the negative brute-force optimum**. It is too large to be the practical solver, which motivates the second formulation.

Formulation 2 · Dicke-state / XY-mixer QAOA (10 qubits, no slack)

Instead of penalising the cardinality constraint, we **build it into the circuit**. The initial state is the uniform superposition of all weight-six bitstrings, and the mixer only swaps excitations between qubit pairs, so the Hamming weight stays exactly six throughout.

$$|D_6^{10}\rangle = \frac{1}{\sqrt{210}} \sum_{|x|=6} |x\rangle, \qquad |\psi(\gamma,\beta)\rangle = \prod_{l=1}^{p} e^{-i\beta_l H_{XY}} e^{-i\gamma_l H_C} |D_6^{10}\rangle$$

- **Cost layer:** one R_Z per qubit with angle proportional to the normalised value $\tilde{v}_i = v_i/\max|v|$.
- **Mixer:** complete-graph XY mixer, an $R_{XX}(\beta)$ followed by $R_{YY}(\beta)$ on each of the 45 qubit pairs.
- **Optimisation:** p ∈ {1, 2, 3, 4} × {COBYLA, SPSA} × 10 restarts = 80 settings, 200 iterations each. A vectorised exact statevector evaluates the expectation during training; the trained circuit is then sampled on Qiskit Aer with 8,192 shots. Restart seed = 42 + 10000·p + (0 for COBYLA, 5000 for SPSA) + restart.

Constraint handling, bitstrings and decoding

- **Cardinality** is guaranteed by the Dicke start and XY mixer (unit-tested to stay in the weight-six sector, and the vectorised objective is tested to match the circuit).
- **Budget and capacity** are checked after decoding every measured bitstring; none are silently accepted.
- **Decoding:** Qiskit prints classical bit 0 on the right, so each count string is reversed to $x_0 \ldots x_9$; bit i then maps to the customer at top-10 rank i+1. The exact optimum is the bitstring **1110100101** (C0081, C0031, C0079, C0222, C0087, C0256).
- **Comparison with exact:** all C(10,6) = 210 subsets are enumerated; the exact optimum is the submitted answer, and QAOA is reported against it.

Repo reference: *src/customer_pipeline/stage2.py* (*_build_qubo*, *qubo_energy_for_selection*, *_qaoa_circuit*, *_vectorized_qaoa_objective*, *_run_dicke_qaoa*, *decode_bits_to_ids*, *solve_selection*); *tests/test_workbook_pipeline.py*.

STAGE 2 · SELECT · RESULTS

The exact optimum, and how close QAOA gets

Customer	p	Cost	Deposit value	Capacity	Segment	Net value v
C0081	0.3983	4	17	1	C	2.771
C0031	0.3719	2	13	1	D	2.834
C0079	0.3323	3	24	3	A	4.976
C0222	0.1605	2	21	1	D	1.370
C0087	0.1561	1	18	1	D	1.810
C0256	0.1528	1	22	4	A	2.361
Total		**13 / 25**		**11 / 22**		**16.122**

All 210 six-of-ten subsets are feasible: the largest possible cost is 18 and the largest capacity 17, so **both limits are inactive for this pool** and the problem reduces to picking the six best net values. C0183 (rank 4) and C0338 (rank 9) are excluded purely on value (0.348 and 0.760), not on constraints.

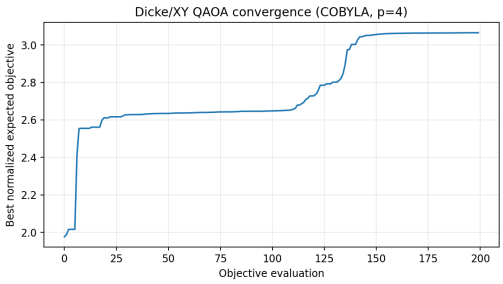


Fig. 4. Best normalised expected objective for the winning run (COBYLA, p = 4). Two jumps, early and near evaluations 110–145, separated by a plateau near 2.6, finish at about 3.07: the optimiser escapes a local plateau.

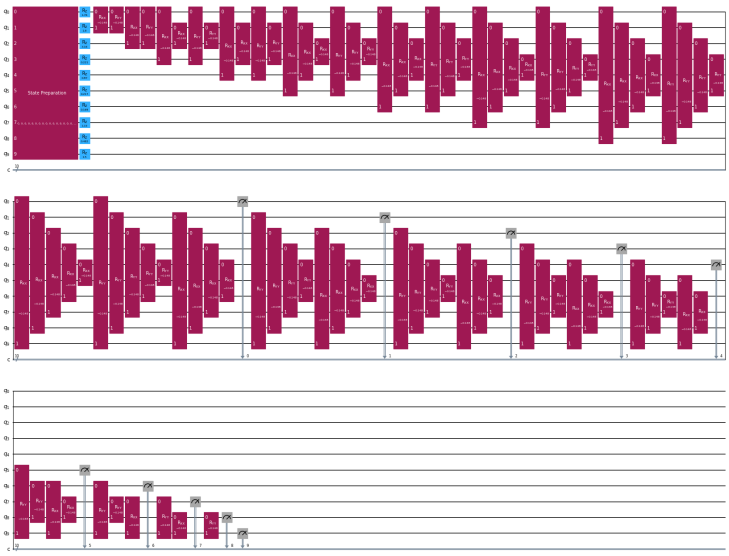


Fig. 5. Compact p = 1 QAOA schematic (logical depth 37): Dicke state preparation, one R_Z per qubit, R_{XX}/R_{YY} ($\beta = -0.148$) on 45 pairs, then measurement. The real p = 4 circuit is far deeper.

QAOA quantity	Value
Best run	COBYLA, p = 4, restart 4, optimiser seed 40046, simulator seed 42; 100% of sampled bitstrings feasible
Probability mass on exact optimum	3,105 / 8,192 = 37.9% (best of 80). Chance for a random subset: 1/210 = 0.48%
Typical run	Median 0.68%; only 3 of 80 settings reach at least 10%
Best sampled objective / gap to exact	16.122 / 0 (absolute and relative); submitted answer gap 0
Transpiled Stage 2 circuit (linear topology)	Depth 5,376; 1,733 CX, 3,753 RZ, 2,168 SX, 147 X

Reading the QAOA result honestly

- QAOA recovers the exact optimum, but the 37.9% figure is the **best of 80 runs, chosen with knowledge of the answer**. The typical run is near chance, so it is not a reliability guarantee.
- The transpiled depth of 5,376 (1,733 CX) includes **generic state preparation of the weight-six Dicke state and routing of the all-to-all XY mixer on a linear topology**, so a dedicated Dicke circuit would be shallower.
- A problem with 210 candidate subsets is trivial to enumerate classically. This stage demonstrates a correct constraint-preserving quantum formulation; **it does not demonstrate quantum advantage**.
- **Robustness:** across 10,000 stress draws that shift all candidate probabilities by a shared calibration-based offset, the submitted six remain the exact optimum in 100% of draws.

Repo reference: *stage2_selection.csv* and *stage2_selection.json* (selected IDs, budget/capacity, objective, measured bitstrings with counts, brute-force optimum, gaps, transpilation); *results/stage2_qaoa_sweep.csv* (all 80 runs); *results/stage2_qaoa_convergence.png*.

STAGE 3 · SIMULATE

The chance that at least three of the six subscribe

Each selected customer gets one qubit. A single R_Y rotation sets the probability of measuring 1 to the customer's Stage 1 probability. Customers are assumed to respond independently, and a shot succeeds when at least three qubits read 1.

$$R_Y(\theta_i)|0\rangle: \quad P(1) = \sin^2(\theta_i/2) = p_i \;\Rightarrow\; \theta_i = 2\arcsin\sqrt{p_i}$$

Qubit order: q0 = C0081, q1 = C0031, q2 = C0079, q3 = C0222, q4 = C0087, q5 = C0256. Qiskit count strings are little-endian (qubit 0 is the rightmost character), and the code reverses each string before counting ones. Because the qubits are independent, the exact answer is a Poisson-binomial tail, computed by dynamic programming:

$$P_n(k) = P_{n-1}(k)\,(1-p_n) + P_{n-1}(k-1)\,p_n, \qquad P(S \geq 3) = \sum_{k=3}^{6} P_6(k)$$

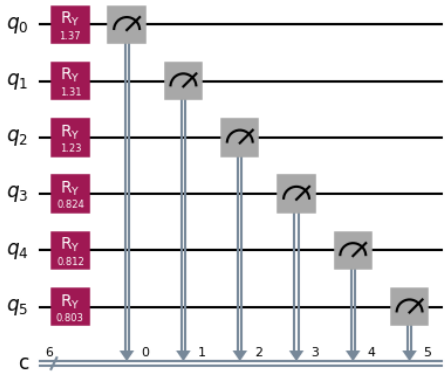


Fig. 6. Stage 3 circuit. Angles 1.37, 1.31, 1.23, 0.824, 0.812, 0.803 rad. No two-qubit gates (depth 5).

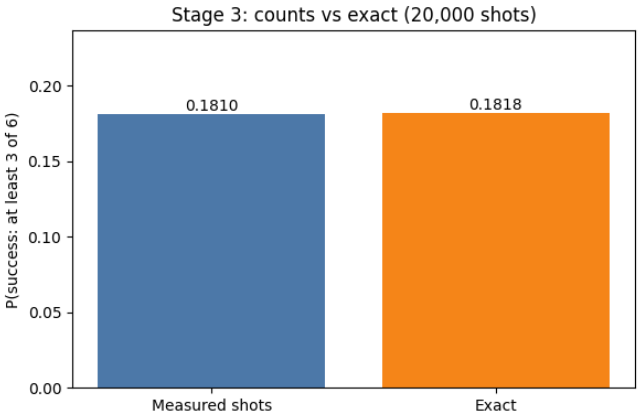


Fig. 7. Measured success fraction (3,620 of 20,000 shots, 0.1810) against exact 0.181754. The gap is ordinary shot noise.

Estimator	P(at least 3 of 6)	Notes
Exact (Poisson-binomial)	**0.181754**	Reference the simulation is judged against
Ideal Aer, 20,000 shots, seed 42	0.1810	95% Wilson 0.1757–0.1864
Independent Bernoulli Monte Carlo, 1,000,000 draws, seed 42	0.181689	95% Wilson 0.180934–0.182446; abs. error 0.000065
Synthetic noise, unmitigated	0.20195	Abs. error 0.02020
Synthetic noise, readout-mitigated	0.19038	Abs. error 0.00863; interval covers shot noise only

Noise, transpilation and uncertainty

- **Noise model (Qiskit Aer, synthetic):** depolarising error 0.005 on single-qubit X/SX, 0.02 on CX, readout assignment errors 0.02 / 0.03. Readout mitigation inverts the tensor-product assignment matrix A:

$$p_{\text{corr}} = A^{-1}\,p_{\text{obs}}$$

- **Transpilation** (linear coupling map, optimisation level 1, seed 42): Stage 3 uses 6 logical / 6 mapped qubits, depth 5, 12 RZ, 12 SX and no two-qubit gates.
- **Sensitivity:** if every customer had the same probability, P(at least 3) would be 0.016, 0.047, 0.099 and 0.169 at 0.10, 0.15, 0.20 and 0.25, against 0.182 for our six. The three high-probability customers drive the result.
- **Propagating Stage 1 uncertainty:** 10,000 draws add one shared offset, sampled from the top-decile calibration interval (−0.130 to +0.161), to all probabilities. The Stage 3 exact probability then has 2.5 / 50 / 97.5 percentiles of **0.0291 / 0.2079 / 0.4899**. This is a group-level proxy, not a customer-specific confidence interval.
- Everything quantum here is simulated: **no IBM hardware or real backend was used**.

Repo reference: *src/customer_pipeline/stage3.py* (*run_simulation*, *exact_at_least_k*); *submission_artifacts.py* (Monte Carlo, noise, transpilation, stress test); *stage3_simulation.csv/.json*; *results/stage3_sensitivity.csv*.

RULES · LIMITATIONS · BUSINESS RECOMMENDATION

Playing fair, knowing the limits, and what the bank should do

Compliance with the challenge rules

Rule / deliverable	How it is met	Where
Never use the test set to select features, tune, calibrate or choose hyper-parameters	Test data is only passed to the final *predict_proba*; guards reject test labels and *duration*	*stage1.py*, *workbook.py*, tests
Fit all preprocessing on training data only	Encoders, scalers, SVD, angle scaling, feature/model selection and Platt calibration sit inside CV training folds	*stage1.py*
Same Stage 1 probabilities in Stages 2 and 3	One ranked prediction table feeds both; tests check IDs and probabilities match	*pipeline.py*, tests
Separate quantum from classical; no unsupported advantage claims	Separate metric rows and sections; classical wins Stage 1; QAOA is a comparison	this report, README
Three result files, one per stage	*stage1_predictions.csv*, *stage2_selection.csv/.json*, *stage3_simulation.csv/.json* at repo root	repo root
Seeds, backend, shots, qubit counts recorded	Master seed 42; table below; full detail in *results/RUN_INFO.md*	*settings.yaml*

Run	Backend	Qubits	Shots	Seeds
Stage 1 ZZ kernel	Qiskit 2.6.2 exact statevector	4 or 6	0 (exact)	42
Stage 2 QAOA	Qiskit Aer 0.18.2 (simulated linear topology)	10	8,192 × 80	optimiser formula above; simulator 42; transpiler 42
Stage 3 ideal / noisy	Qiskit Aer / Aer with synthetic noise	6	20,000 each	simulator 42; transpiler 42
Monte Carlo	NumPy	–	1,000,000	42

Reproduce with Python 3.13: *pip install -r requirements.txt*, *python -m unittest discover -s tests*, *python main.py*. Versions are pinned and MLflow logs locally; no cloud account is needed.

Limitations

- **Weak signal.** Only 35 positive labels. Logistic ROC-AUC is 0.619 with a bootstrap interval of 0.515–0.717, so the lower end is barely better than chance.
- **Top ten is shaky.** Only three test customers separate clearly; ranks 4–11 differ by tiny margins, and Precision@10 reflects a single feature.
- **Scenario inputs.** Costs, deposit values and capacity come from the workbook, not measured operations; here the limits never bind.
- **Easy optimisation.** 210 subsets are trivial to brute-force; QAOA's best-of-80 result is selection-biased and the median run is near chance.
- **Simulation only.** Noise is synthetic, mitigation ignores calibration uncertainty, and the quantum kernel (alignment 0.034) was evaluated by exact statevector.
- **Modelling assumptions.** Stage 3 assumes independent responses with calibrated probabilities; the stress test is a shared-offset proxy; repeated-CV intervals are too narrow; test outcomes are unknown.

Business recommendation

From the selected six (derived from the submitted probabilities): expected subscribers ≈ **1.57 of 6**; chance that at least one subscribes ≈ **85%**; at least three ≈ **18%**; expected deposit value ≈ 29.1 against contact cost 13, which is the 16.122 net objective in workbook units, not guaranteed revenue.

- **Run a controlled pilot,** not a full rollout: contact the six, measure real response and deposit value, and plan for one or two subscribers rather than three or more.
- **Prioritise C0081, C0031 and C0079.** They are the only customers clearly above the rest (0.33–0.40 vs about 0.15). Keep ranks 4–11 as a reserve list, since they are nearly interchangeable.
- **Add a holdout group** of similar uncontacted customers so campaign uplift is measured rather than assumed.
- **Collect more labels before adding model complexity,** then retrain and recalibrate; replace scenario costs and capacity with real figures (the exact solver already handles binding limits).
- **Use the classical optimiser in production.** Revisit quantum optimisation only for problems orders of magnitude larger, against a fair classical baseline.

Conclusion. The pipeline is leak-free, reproducible and honest: a calibrated logistic model ranks the customers, an exact search picks six within the limits, QAOA reproduces that optimum, and a six-qubit circuit matches the exact subscription probability to within shot noise. Full code, outputs and notebook: github.com/souvikbasuroy/Qampaign.